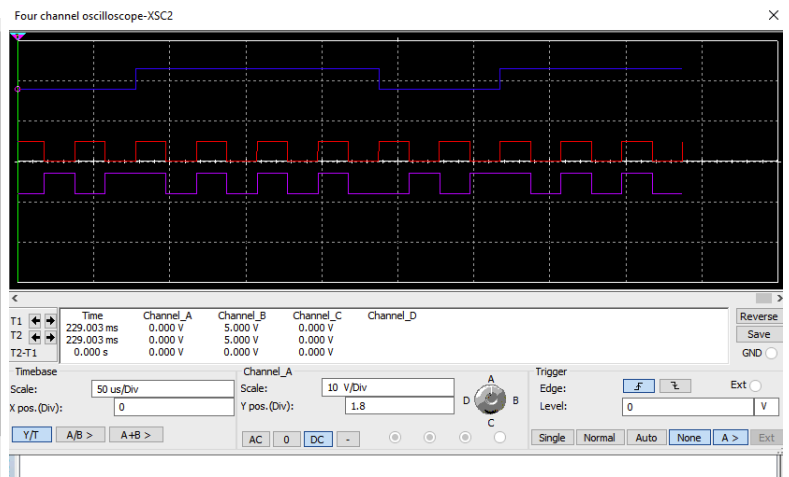
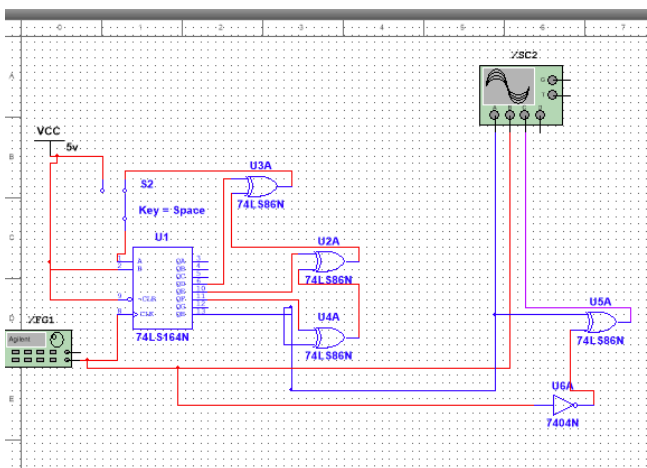
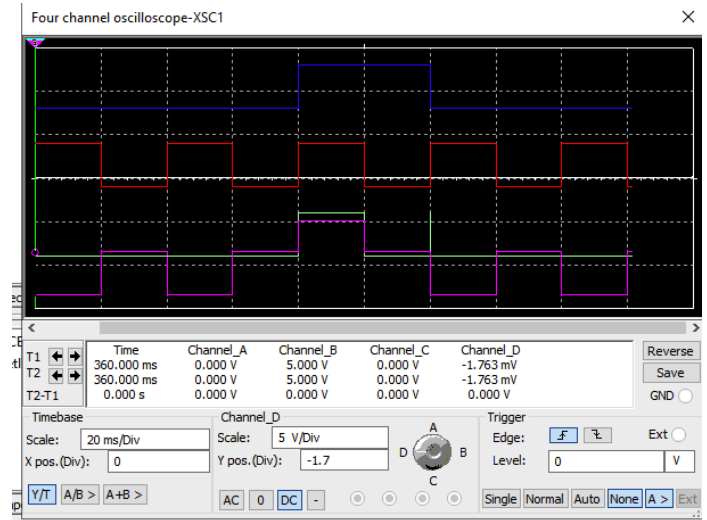
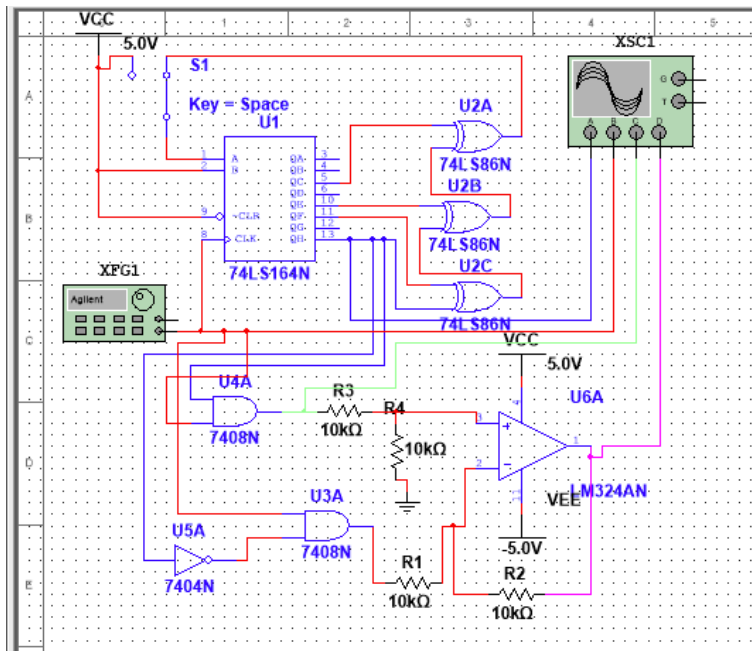


ETE 4351L Data Communication and Networking
 Lab 9: Line Codes
 Mariana Rivera, Sarah Boyd, Fatima Saldaña-Gutierrez
 Professor Jason Mandala
 March 24, 2026



OBJECTIVES

- Create a pseudo-random data (PRNG) generator using a Linear Feedback Shift Register (LFSR)
- Create a circuit to generate a Manchester encoded data stream
- Create a circuit to generate NRZ, Unipolar RZ, and Bipolar RZ encoding schemes
- Practice using Multisim circuit simulation software

INTRODUCTION

Line codes are various methods of representing data before modulation; in order, data, coding (where line codes are used), pulse shaping, and then modulation. Different line codes have different aspects, such as required bandwidth, amount of DC component, and ease of synchronization. The goals for line codes are low bandwidth, no DC component (average value near zero), and having pulses at regular intervals regardless of exact data being sent.

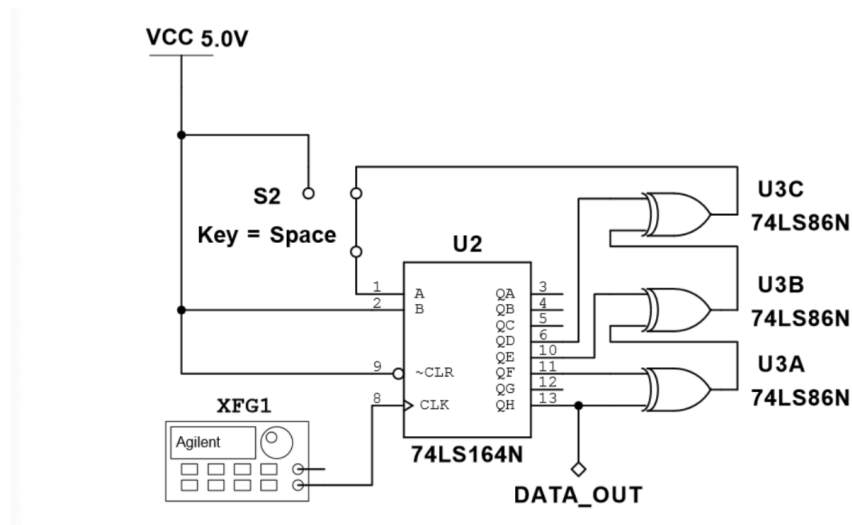
We want a low required bandwidth to represent the data. Since this data will be used to modulate a carrier, we want to use as little as possible. DC voltage is the average value of the waveform over a long time. It is often assumed that the data 1 and 0 are equally likely and occur equally as much, but this is not true. The DC component can be an issue if we are using a system that doesn't pass DC. Sometimes the circuit has a coupling capacitor which blocks dc, or sometimes the signal is transformer coupled. Additionally, transmitting DC can lead to unnecessary power loss. Pulses at regular intervals are also important- the receiver must know the clock phase of the data that is being sent so that it can synchronize and then make sure that the data is being sampled at the correct time. Therefore clock recovery is a common part of most receivers. If there are no pulses at regular intervals the receiver might lose synchronization with the data stream.

There are many different types of line codes. Line codes can be unipolar or bipolar. Unipolar uses voltages of positive to zero where bipolar uses positive and negative. Bipolar has a nice advantage in that the DC component of the signal is reduced. Unipolar has the advantage of simplicity and lowering the bandwidth requirements. Another line coding scheme is NRZ and RZ. Line codes can be non-return to zero (the voltage not returning to zero each bit period) or

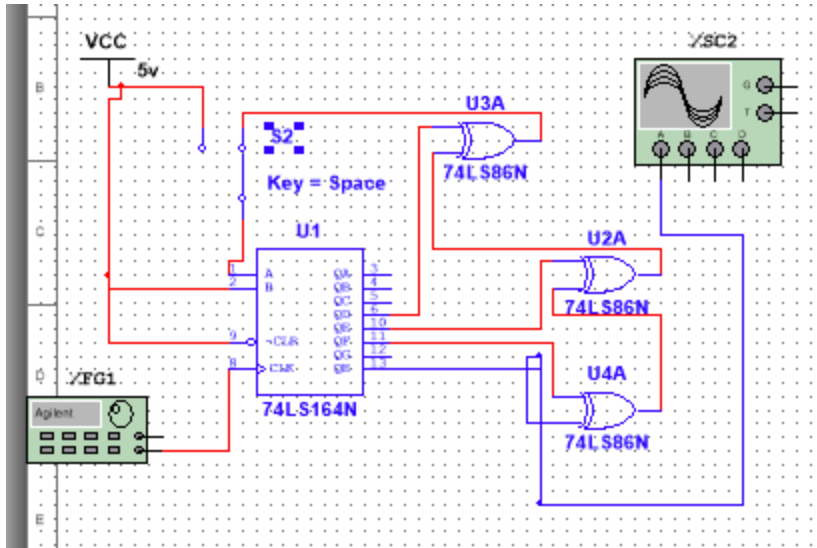
return to zero (the voltage returns to zero each pit period regardless of the data being sent). For NRZ there are low bandwidth requirements compared to other types of line codes, but there could be long streams of 0's and 1's which make synchronization difficult. For RZ, the bandwidth requirements are increased due to more transitions, and it is still susceptible to long streams of the same bit causing synchronization problems. However, the DC component is reduced. Additionally, a Manchester coding scheme can be used, where there is a transition from low to high or high to low in the middle of each bit period regardless of the data being sent. The bandwidth is higher compared to NRZ, and there is a non zero DC level, but there are regular transitions and there is a guaranteed transition each bit period. There is also differential manchester code, NRZ-inverted, and AMI (alternate mark inversion). A chart with different types of line codes, consisting of information from **Professor Jason Mandala's** notes, can be seen later in this report.

RANDOM DATA GENERATOR

1. Build the circuit of Figure 1 using Multisim. The function generator should create a clock signal of 0 to 5 V, at a frequency of 25 Hz.

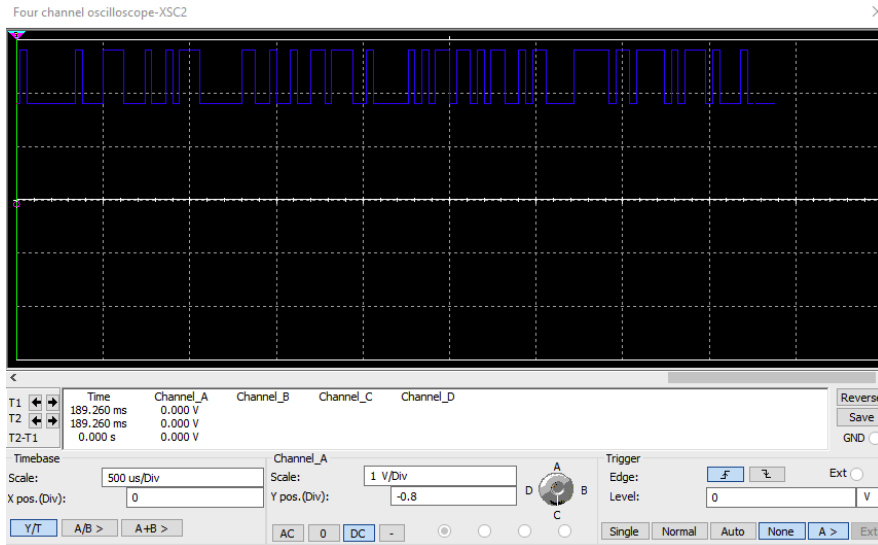


IMG 1. Linear feedback shift register



IMG 2. *Random data generator circuit*

2. Measure the output “DATA_OUT” with an oscilloscope.
3. Begin with pin A of the shift register connected to the XOR feedback.
4. Move the connection to pin A temporarily to +5V.
5. Return the connection back to the XOR feedback position.
6. Verify your data stream is working on the oscilloscope.



IMG 3. *Data from random number generator, verified on the oscilloscope*

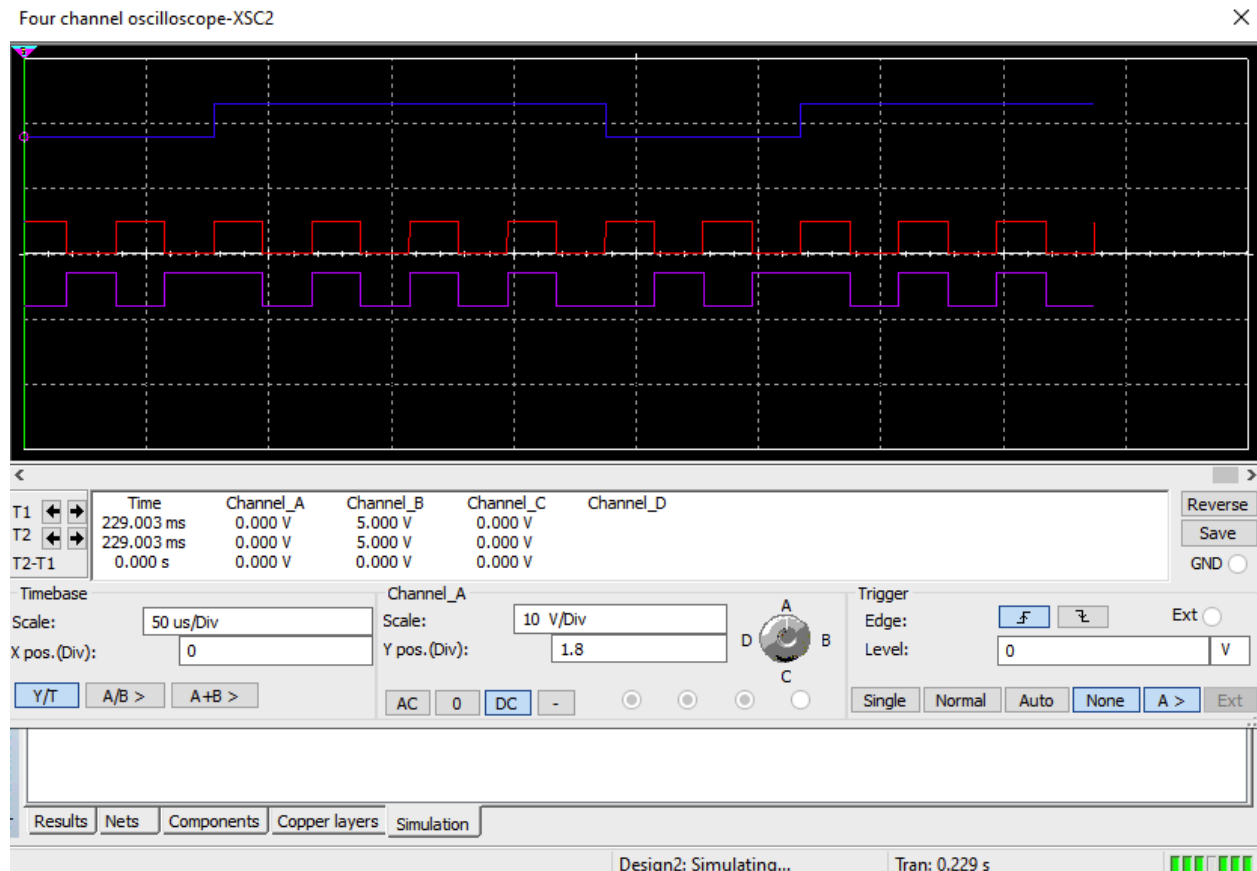
MANCHESTER ENCODING

-
- ```

graph LR
 DATA --> U3D
 CLK --> U1A
 U1A --> U3D
 U3D --> MANCHESTER
 subgraph ICs
 U1A["7404N"]
 U3D["74LS86N"]
 end

```

5



**IMG 6.** Manchester encoding circuit output. Blue is from the data, red is from the clock, and purple is the output with the manchester encoding scheme.

4. Capture screenshot and verify output signal agrees with theory.

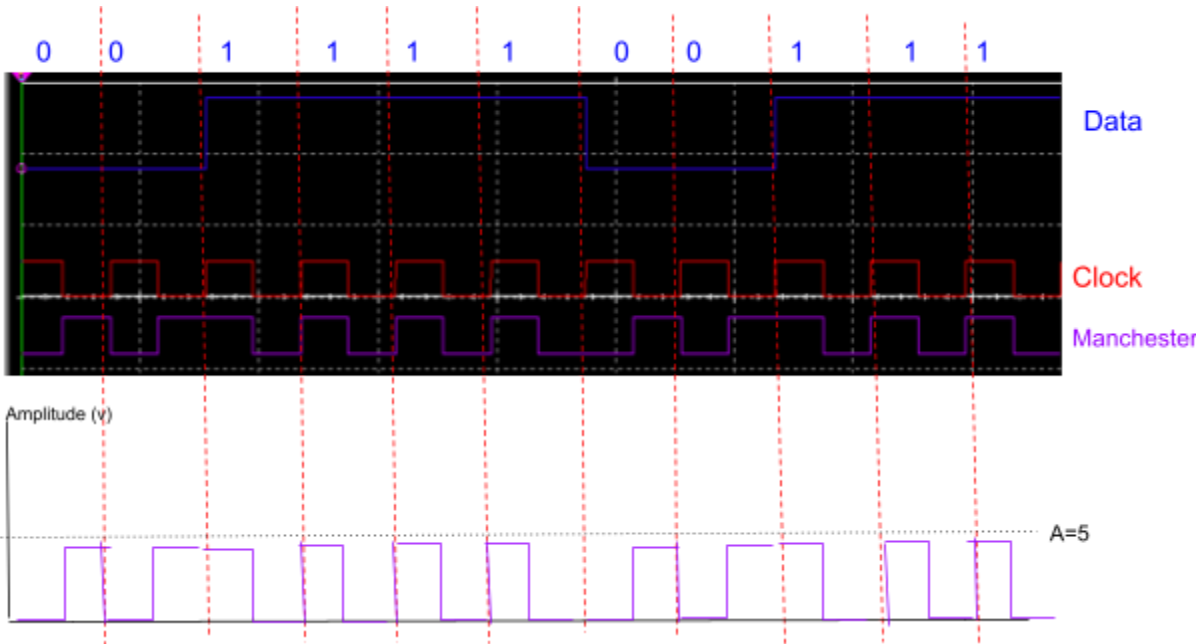
The Manchester signal (purple) is correct because it shows a transition in the middle of every bit period. A key attribute of the Manchester coding scheme is that there is a transition from low to high or high to low in the middle of each bit period regardless of the data being sent. When you compare the purple signal to the blue data signal, you can see that for each bit the purple waveform flips once half way through the bit and that the direction of that transition changes depending on whether the data is a 1 or 0. This matches the rule from the handout that Manchester encoding requires a mid bit transition and encodes data using a high to low or low to high change. The fact that the transitions are aligned with the clock (red) and occur consistently for every bit confirms that the encoding is functioning properly.

To check, we can draw the signal ourselves and cross check with the purple output signal.

Rules:

“1” bit has a transition from high to low, in the middle of the bit period

“0” bit has a transition from low to high, in the middle of the bit period



**IMG 7.** Drawing of Manchester encoded signal for this lab

Since the drawing using the manchester encoding scheme rules and the actual manchester output matches, we can assume that the results on the scope are accurate.

5. In your report, comment on the advantages and disadvantages (BW, DC bias, etc.) of the Manchester encoding scheme.

Manchester encoding has several advantages and disadvantages. One advantage is that it always contains a transition in the middle of every bit, which makes clock recovery easier because the timing information is built into the signal itself. However, because of the increase in transitions and the sharpness of these transitions, there is a higher requirement for frequency content resulting in a higher bandwidth. This higher bandwidth is a disadvantage, and is higher than NRZ (non return to zero). However, the transitions help reduce the DC component

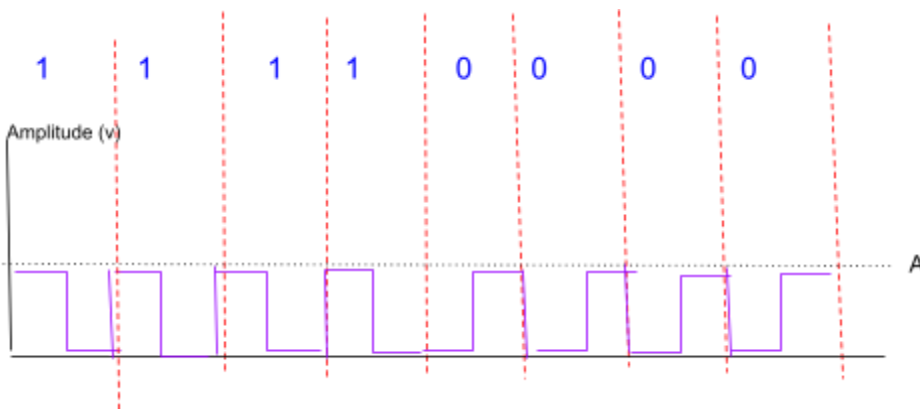
compared to NRZ encoding, which is beneficial since some communication channels cannot pass DC. It is important to note that the DC level is still not zero, however,

6. In your report, draw (by hand if you want) the resulting Manchester encoded signal for data “11110000” and “10101100”.

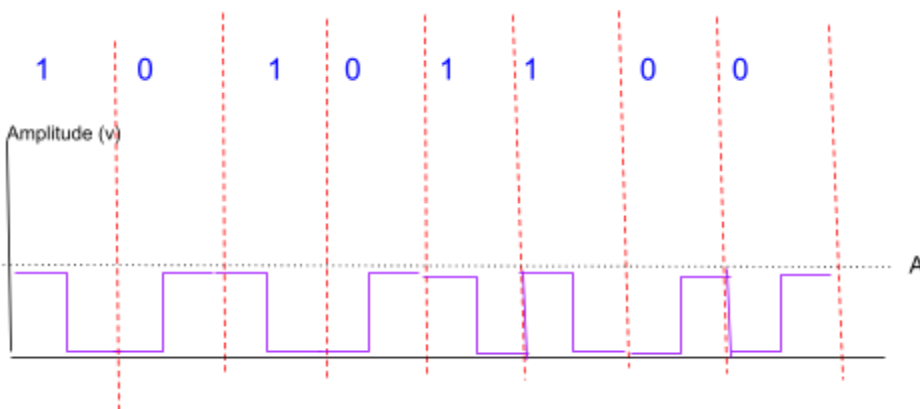
Rules:

“1” bit has a transition from high to low, in the middle of the bit period

“0” bit has a transition from low to high, in the middle of the bit period

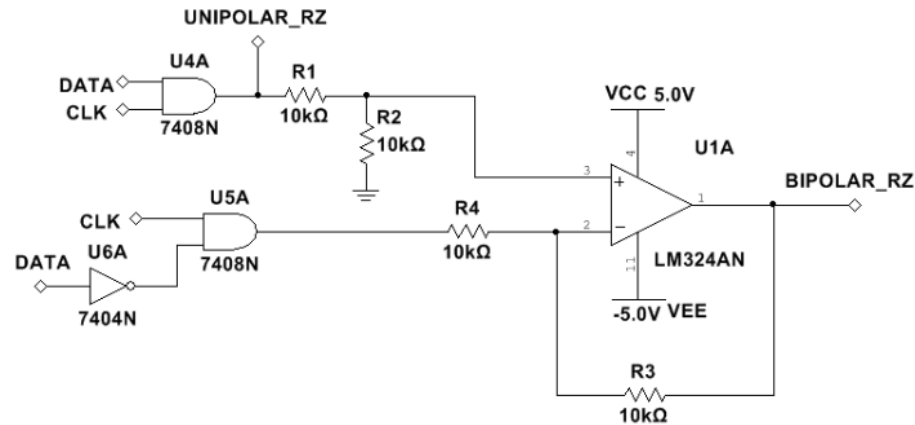


**IMG 8.** Drawing of Manchester encoded signal for “11110000”

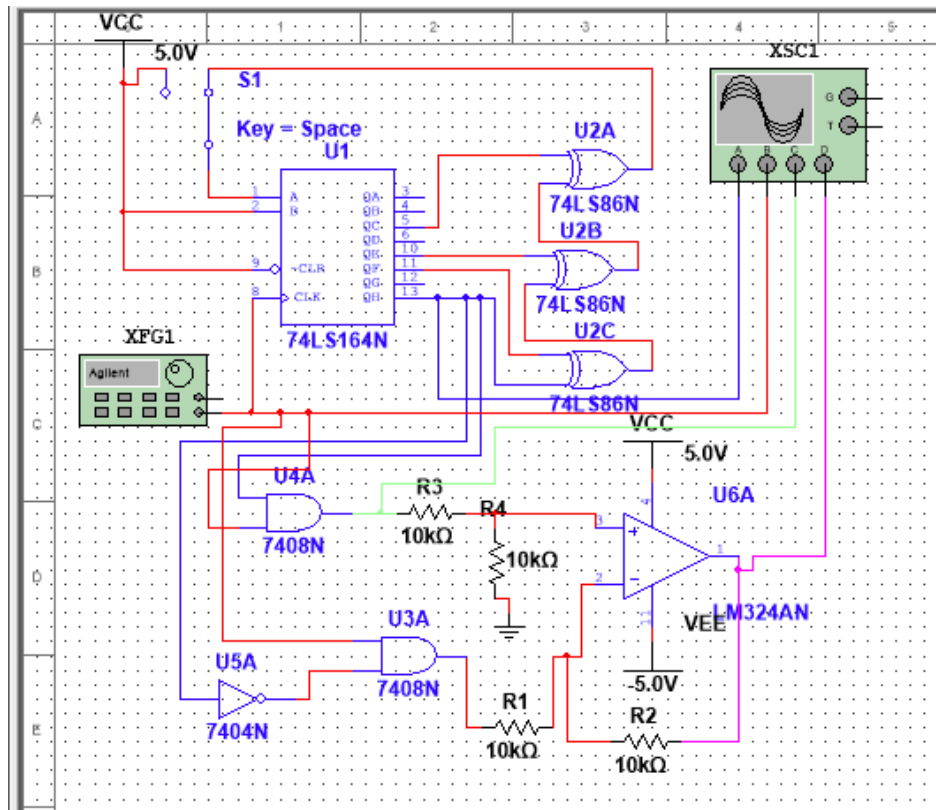


**IMG 9.** Drawing of Manchester encoded signal for “10101100”

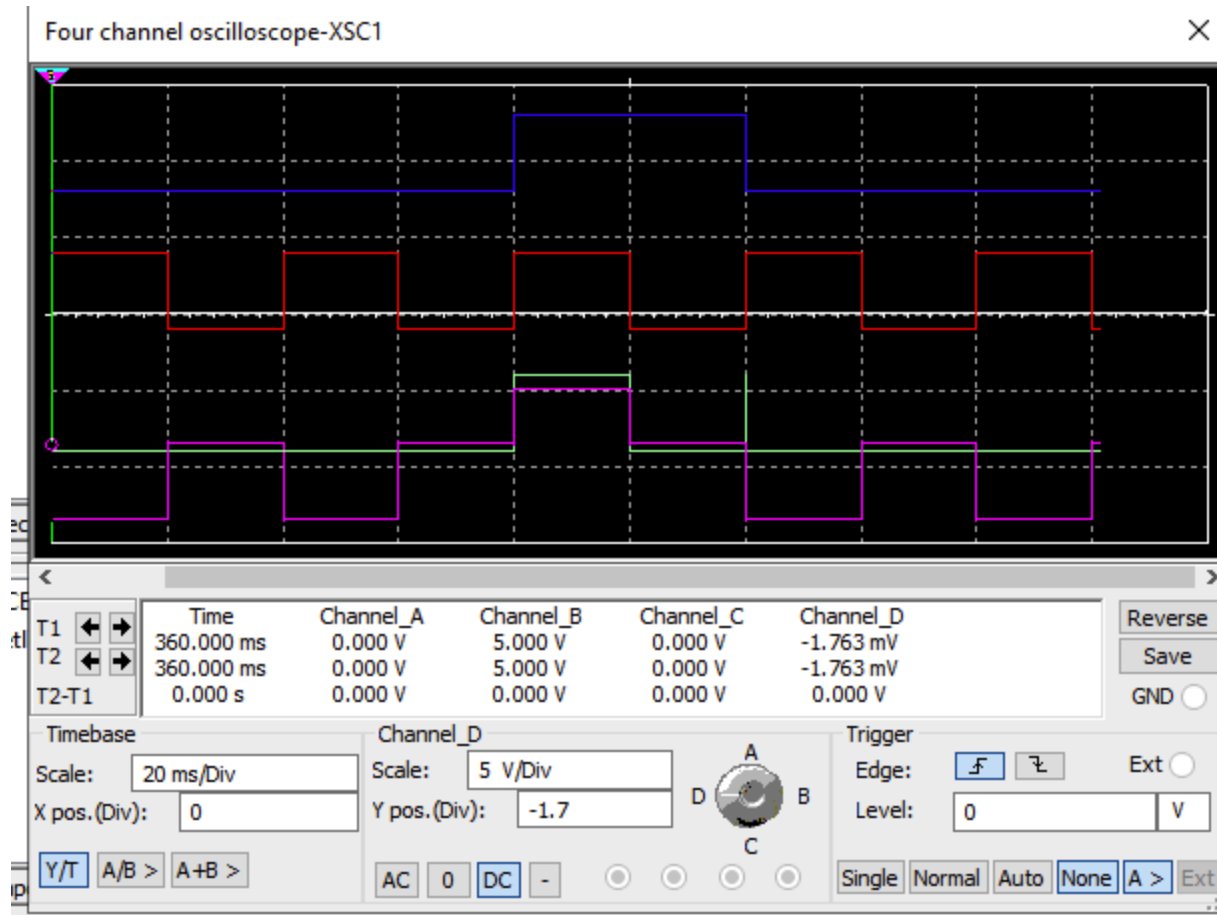
1. Remove the Manchester encoding circuit from your board. Add the circuit of Figure 10 to your board. Connect the “DATA\_OUT” from before to the “DATA” port of the new circuit.



**IMG 10.** *Bipolar and unipolar RZ circuit*

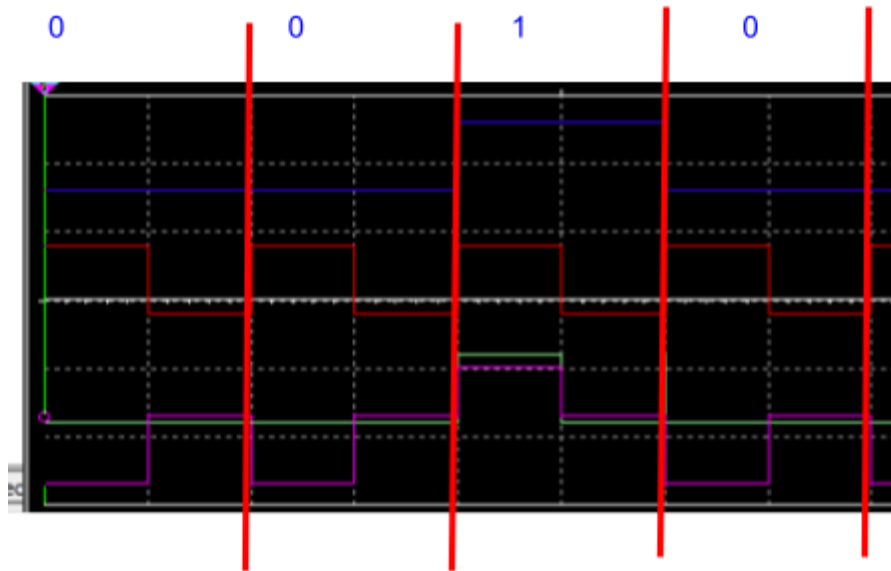


**IMG 11.** *Bipolar and unipolar RZ circuit*



**IMG 12.** Bipolar and unipolar RZ circuit. Green is unipolar, pink is bipolar, red is the clock signal, and blue is the data.

2. Connect the “CLK” to the same clock signal as your PRNG circuit.
3. Measure the “UNIPOLAR\_RZ” and “BIPOLAR\_RZ” outputs on the oscilloscope. If you are not getting the correct output, check and make sure your PRNG is working first, then check that you are getting “UNIPOLAR\_RZ” correctly. Next check that you are getting inverted “DATA” at the input of AND gate U5. If this is all correct, then check the wiring of the LM324.
4. Capture screenshot with PRNG data and RZ output signal(s) simultaneously, and verify output signal agrees with theory. In other words, connect Ch. 1 to PRNG data, and Ch. 2 to the RZ encoded data for side by side comparison.



**IMG 13.** *Confirmation of results*

We can see the first bit randomly generated is 0 (as seen from the blue signal). The red is the clock wave, defining the symbol time (one clock cycle is one symbol time). In this system there is one bit transmitted per clock cycle and there is one bit per symbol ( $M = 2^1$ , so 1 bit per symbol). Observing the unipolar line coding scheme behavior (green signal) at 0 the signal stays at 0 but with the data 1 it returns to 0 half way through. We can see that for the bipolar line code scheme, a 0 has a negative amplitude and returns back to 0 half way through the pulse. Also, for the data 1, the signal returns back to 0 half way through the pulse.

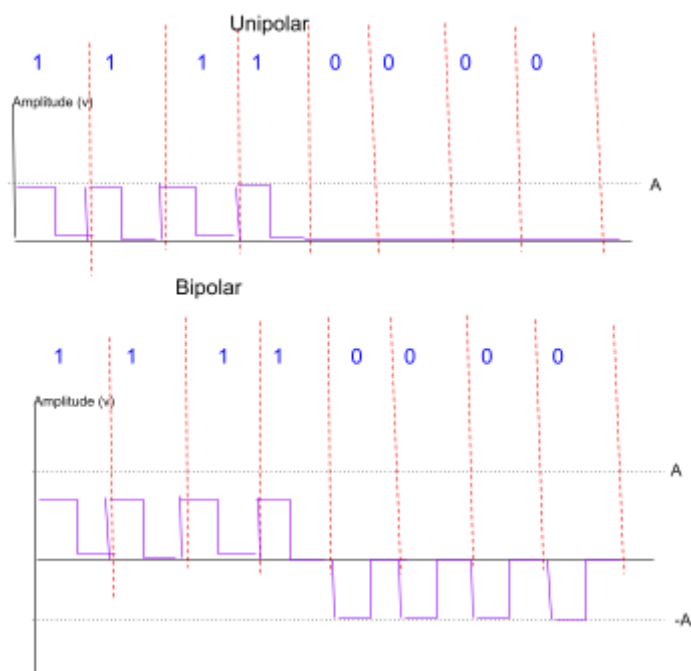
Therefore, the output signal agrees with theory.

5. In your report, comment on the advantages and disadvantages (BW, DC bias, etc.) of the Unipolar and Bipolar RZ encoding scheme.

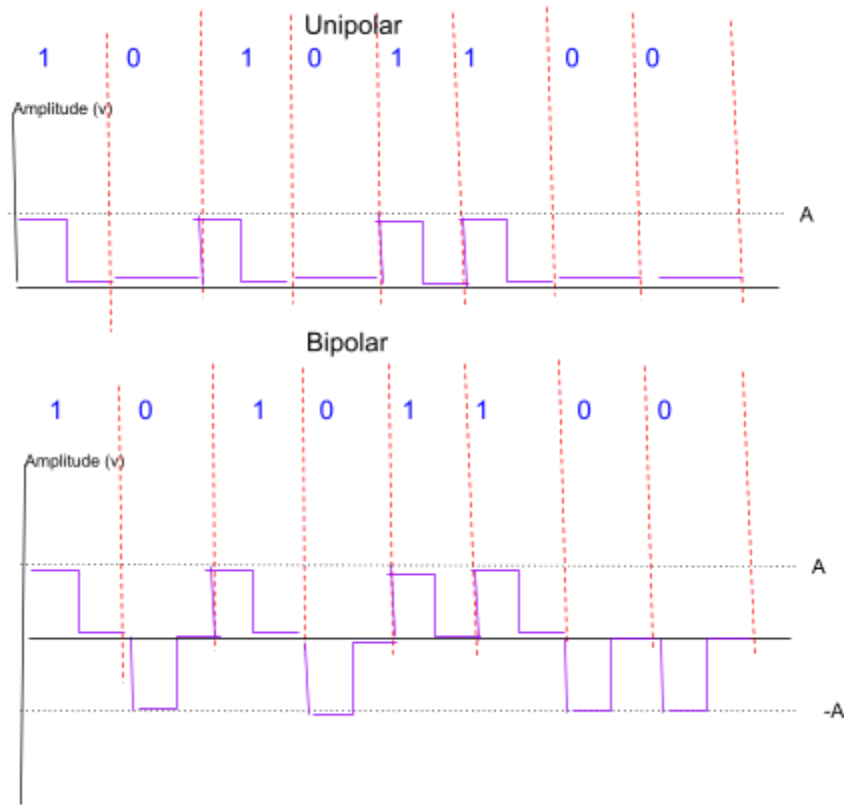
For RZ bipolar, there is increased bandwidth requirements due to transitions in the middle of the bit period. The DC component is zero, assuming that there is an equal number of 0s and 1s sent. Since this is likely not the case as the amount is not exactly even, the DC component is probably approximately zero. Synchronization is improved due to there being a pulse happening every bit period. For RZ unipolar, there is also increased bandwidth

requirements (doubled) due to transitions in the middle of the bit period. Unfortunately, this scheme is still susceptible to long strings of the same bit without pulses always happening every bit period, which can cause synchronization problems. The DC component is reduced by a factor of 2x, yet is still not quite zero.

6. In your report, draw the resulting Unipolar and Bipolar RZ encoded signals for “11110000” and “10101100”.



**IMG 14.** *Drawing of unipolar and bipolar signal for “11110000”*

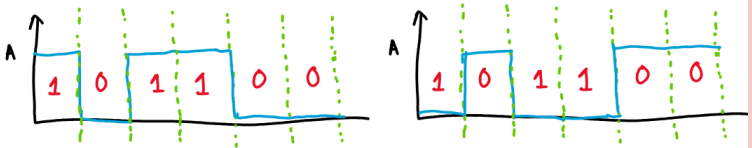
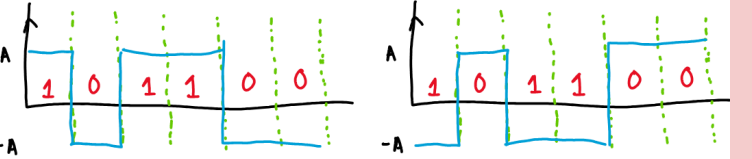
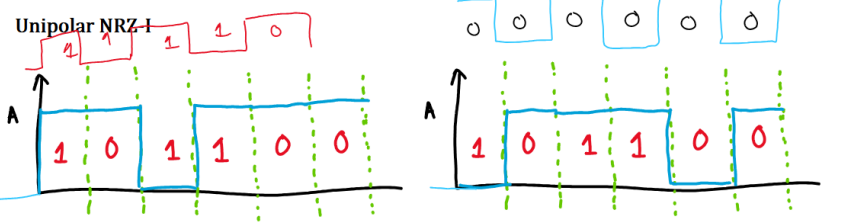


**IMG 15.** *Drawing of unipolar and bipolar signal for “10101100”*

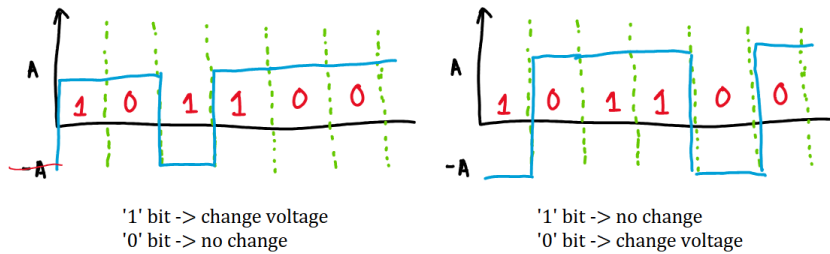
## Conclusion

In this lab, several line coding techniques were implemented and compared on Multisim, including NRZ, Manchester encoding and unipolar and bipolar RZ. The linear feedback shift register circuit was used to generate a digital data stream with random values, which was then encoded using different methods. The results were observed to be successful for Manchester, Unipolar RZ, and Bipolar RZ. The results on the scope match the conceptual analysis and theory learned in class. The results also demonstrate that NRZ is simple but suffers from DC bias and lack of transitions, while Manchester encoding improves clock recovery by ensuring a transition in every bit but requires greater bandwidth. RZ encoding also increases transitions and reduces DC components, with bipolar RZ further minimizing DC bias. Overall, this lab demonstrated the trade-offs between bandwidth, DC content, and getting regular pulses in digital communication systems.

Below is a helpful chart of the key line codes we learned about in lecture, using the notes taken by Professor Jason Mandala. This chart was used as a reference for this lab.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>NRZ - Non Return to Zero or NRZ-L.</b> In NRZ-L, the voltage does <i>not</i> return to zero each bit period</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <p><b>Unipolar NRZ (aka Unipolar NRZ-L)</b> - One bit is defined at +A volts, while the other bit is defined as zero volts.</p>                                                                                                                                                                                         |  <p>'0' bit -&gt; zero volts<br/>'1' bit -&gt; +A volts<br/>Commonly used basic representation of data</p> <p>'0' bit -&gt; +A volts<br/>'1' bit -&gt; zero volts<br/>Note: this is not used in any commo systems, but meets the definition of NRZ-L.</p> |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <p><b>Bipolar NRZ (aka Bipolar NRZ-L)</b> - One bit is defined at a certain voltage "level" (either positive or negative), the other bit is defined for another level (also positive or negative).</p>                                                                                                                  |  <p>'1' bit is +A volts, '0' bit is -A volts</p> <p>'1' bit is -A volts, '0' bit is +A volts<br/>RS-232 uses this type of line coding s</p> <p>iscussion for NRZ-L: <span style="float: right;">MAX 232</span></p>                                        |
| <p><b>Discussion for NRZ-L:</b></p> <ul style="list-style-type: none"> <li>• <b>NRZ-L Unipolar</b> - Has advantage of simplicity and low bandwidth requirements relative to other types of line codes. Disadvantages are that it has a DC component, and long strings of zeros or ones (e.g. 1111111 or 00000000) cause no pulses to be generated on the line, thus making synchronization difficult .</li> <li>• <b>NRZ-L Bipolar</b> - Very similar to unipolar pros and cons, except that the negative pulse will require more complex circuitry or negative supply, and this also helps reduce the DC component of the signal.</li> </ul> |                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                             |
| <p><b>NRZ-I or NRZ-Inverted</b>, is a line coding scheme in which one bit is represented by a change in voltage, and the other is represented by no change in voltage.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |  <p>Unipolar NRZ-I</p> <p>'1' bit -&gt; change voltage<br/>'0' bit -&gt; no change<br/>Also called NRZ-M (Mark)</p> <p>'1' bit -&gt; no change<br/>'0' bit -&gt; change voltage<br/>Also called NRZ-S (Space). Used in USB prot</p> |                                                                                                                                                                                                                                                                                                                                             |

Bipolar NRZ-I

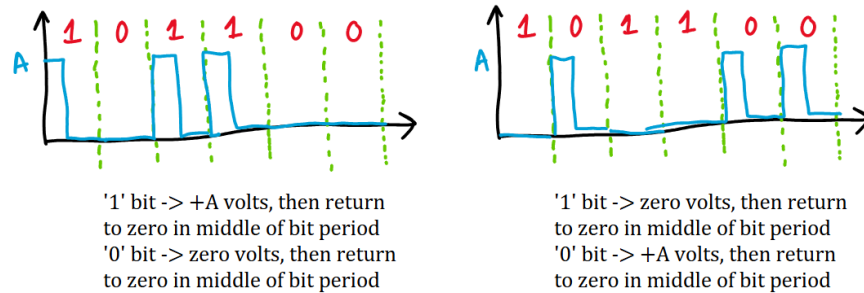


#### Discussion for NRZ-I:

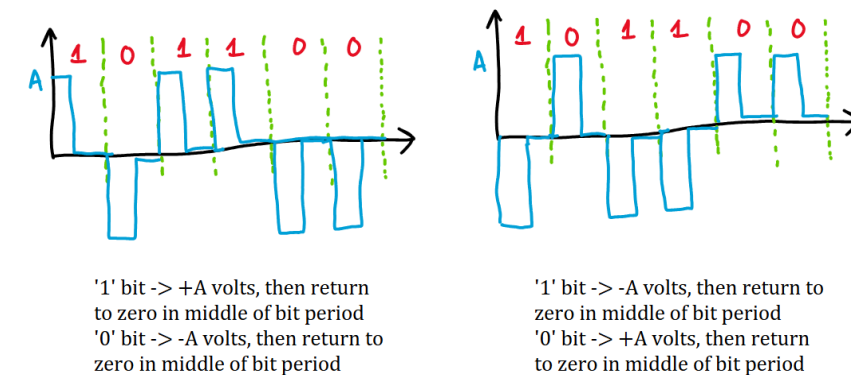
- **NRZ-I Unipolar** - Improves synchronization since sending several ones (in picture below) in succession would now toggle the line up and down, but for sending several zeros in a row there would be no transitions. Bandwidth and DC components are similar compared to NRZ-L.

**RZ - Return to Zero** - For an RZ line coding scheme, the voltage "returns to zero" each bit period, regardless of data being sent.

RZ Unipolar

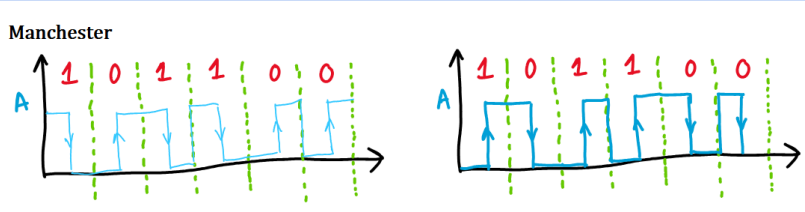
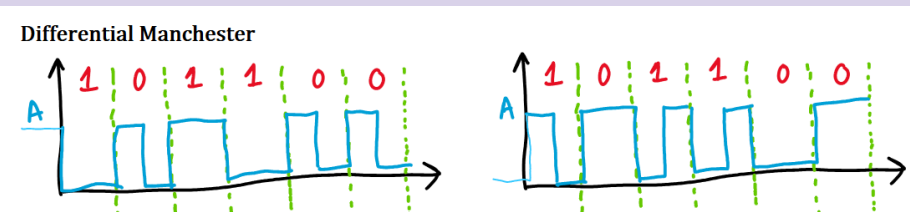
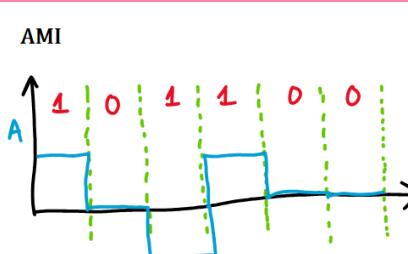
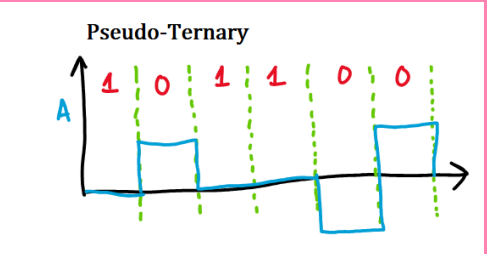


RZ Bipolar



#### Discussion for RZ:

- **RZ Unipolar** - Increased bandwidth requirements (2x) due to transitions at the middle of the bit period. Still susceptible to long strings of the same bit causing synchronization problems. DC component reduced by a factor of 2x, but still not zero.
- **RZ Bipolar** - Increased bandwidth requirements (2x) due to transitions at the middle of the bit period. DC component is zero if an equal number of 1's and 0's are sent. Synchronization improved because there is some kind of pulse happening every bit period.

|                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <p><b>Manchester code (aka bi-Phase)</b> -For Manchester coding scheme, there is a transition from low-to-high or high-to-low in the middle of each bit period, regardless of data being sent. Note: this requires a transition at the edge of the bit period sometimes, depending on the previous data sent.</p>                            | <p><b>Manchester</b></p>  <p>'1' bit -&gt; Transition from high-to-low in middle of bit period<br/>'0' bit -&gt; Transition from low-to-high in middle of bit period</p> <p>'1' bit -&gt; Transition from low-to-high in middle of bit period<br/>'0' bit -&gt; Transition from high-to-low in middle of bit period</p>                                                                |  |
| <p><b>Discussion for Manchester-</b><br/>Bandwidth - higher compared to NRZ<br/>DC Level - non-zero, so not great<br/>Regular transitions - yes, this scheme is good because it guarantees at least one transition every bit period.</p>                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |  |
| <p><b>Differential Manchester code</b><br/>- For differential Manchester code, information is encoded in the presence or absence of a transition at the middle of the bit period. In this scheme, there is <b>always</b> a transition at the edges of the bit period, but in the middle, there is only a transition for one of the bits.</p> | <p><b>Differential Manchester</b></p>  <p>'1' bit -&gt; No transition in middle of bit period<br/>'0' bit -&gt; Transition in middle of bit period<br/>Note: always a transition at edges of bit period.</p> <p>'1' bit -&gt; Transition in middle of bit period<br/>'0' bit -&gt; No transition in middle of bit period<br/>Note: always a transition at edges of bit period.</p>     |  |
| <p><b>Alternate Mark Inversion (AMI)</b> - Each Mark ('1' bit) is inverted compared to the previous one. Bit '0' is encoded as zero volts always.</p>                                                                                                                                                                                        | <p><b>AMI</b></p>  <p>'1' bit -&gt; +A volts if previous '1' was -A volts,<br/>-A volts if previous '1' was +A volts<br/>'0' bit -&gt; zero volts</p> <p><b>Pseudo-Ternary</b></p>  <p>'1' bit -&gt; zero volts<br/>'0' bit -&gt; +A if previous '0' was -A,<br/>or -A if previous '0' was +A</p> |  |

**Table 1.** Line coding information, from Professor Jason Mandala's notes